

Labwork 4: Threads

Duong Tan Binh
Student ID: 2540007

1 Method

I use the same 512 by 600 image and grayscale formula as Lab 3. The CPU loop and 1D kernel stay the same. In the 2D kernel, I calculate both x and y from the thread and block indices. Each thread reads `src[y, x]` and writes one gray value to `dst[y, x]`. I check both coordinates to avoid going outside the image. I use 2D blocks of 8 by 8, 16 by 16 and 32 by 32 threads. The grid size is calculated separately for the width and height, rounding up in each direction.

2 Results

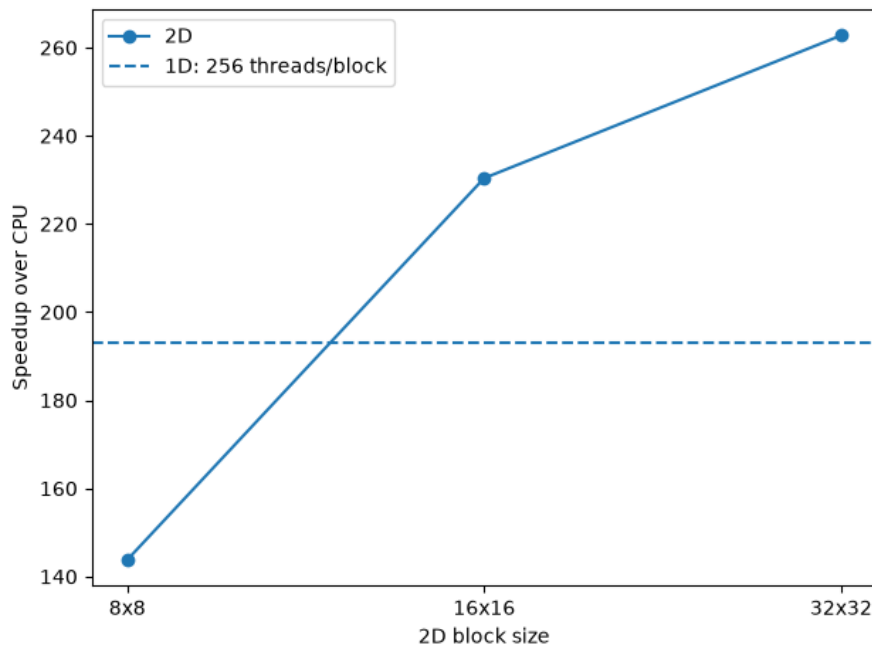
I measure each configuration once with `time.time()`. Both kernels run once before timing to exclude compilation. GPU time includes transfers in both directions. Speedup is CPU time divided by GPU time. I rerun the 1D version with 256 threads per block, using the same image and CPU time for comparison.

```
CPU time: 98.982 ms
1D block size: 256
1D time: 0.513 ms
1D speedup: 193.1
2D block size: (8, 8)
2D time: 0.688 ms
2D speedup: 143.8
2D block size: (16, 16)
2D time: 0.43 ms
2D speedup: 230.39
2D block size: (32, 32)
2D time: 0.377 ms
2D speedup: 262.76
```

The CPU, 1D and 2D versions give the same grayscale values. Below are the input image, CPU result and 2D GPU result.



3 Comparison



The 32 by 32 block is fastest in this run, with a speedup of 262.76, compared with 193.10 for the 1D version. The 8 by 8 block is slower than 1D, so using 2D blocks does not always make this code faster. These are single measurements and may change between runs.

4 Exercise 1

I choose 16 by 16. Each block has 256 threads, so four blocks can fill the limit of 1024 threads per SM, or 32 warps. With 8 by 8, eight blocks give only 512 threads, or 16 warps. A 32 by 32 block has 1024 threads and exceeds the given limit of 512 threads per block. This limit is for the GPU in the question; the RTX 5060 Ti used for the experiment supports 1024 threads per block.

5 Exercise 2

I choose 512 threads per block. Three blocks give 1536 threads per SM. With 128 threads, four blocks give 512 threads. With 256 threads, four blocks give 1024 threads. With 1024 threads, only one block fits, giving 1024 threads.

6 Exercise 3

Four blocks are needed because 2000 divided by 512 must be rounded up. The grid has 4 times 512 = 2048 threads. Only 2000 threads process data; the remaining 48 threads must skip the calculation.